

Problem-83: อธิบาย Minimum Cost Path อย่างละเอียด

ไม่เป็นไร! ผมจะอธิบายให้เข้าใจง่ายๆ ทีละส่วน TH

🎯 ปัญหาคืออะไร?

นิยาม: คุณอยู่ที่มุมบนซ้าย (เริ่มต้น) ต้องไปมุมล่างขวา (จุดหมาย)

- ✅ เดินได้แค่ ขวา หรือ ลง เท่านั้น
- ⚠️ ทุกช่องมีค่าใช้จ่าย (ต้องเสียเงินตอนเข้าห้อง)
- 🎁 ต้องหา เส้นทางที่เสียเงินน้อยที่สุด

ตัวอย่าง:

```
Grid:
[1, 3, 1]    ← เริ่มที่นี่ (1)
[1, 5, 1]
[4, 2, 1]    ← จุดหมายคือที่นี่ (1)
```

เส้นทางที่เหมาะสม: 1 → 3 → 1 → 1 → 1 = 7 บาท

📝 อธิบายโค้ด ทีละบรรทัด

1 บรรทัดแรก: ตรวจสอบปัญหา

```
if not grid or not grid[0]:
    return 0
```

ความหมาย:

- if not grid = ถ้า grid ว่าง (ไม่มีข้อมูล)
- or not grid[0] = หรือ grid[0] ว่าง (ไม่มีแถวแรก) ถ้าเป็นเช่นนั้น คืนค่า 0 (ไม่มีอะไรต้องคำนวณ) เหตุผล: ถ้าไม่มี grid ก็ไม่สามารถหาเส้นทางได้ ป้องกันโปรแกรม crash

2 เตรียมข้อมูล: นับแถวและหลัก

```
rows = len(grid)    # นับแถว
cols = len(grid[0]) # นับหลัก (จำนวนช่องในแถวแรก)
```

ตัวอย่าง:

```
grid = [
    [1, 3, 1],    ← แถวที่ 0
    [1, 5, 1],    ← แถวที่ 1
    [4, 2, 1]     ← แถวที่ 2
]

rows = 3  (มี 3 แถว)
cols = 3  (แต่ละแถวมี 3 หลัก)
```

3 สร้าง "บัญชี" หรือ "ตาราง"

```
dp = [[0 for _ in range(cols)] for _ in range(rows)]
```

ความหมาย: สร้างตาราง 2 มิติที่ว่างไว้ เช่น:

```
dp = [
    [0, 0, 0],
    [0, 0, 0],
    [0, 0, 0]
]
```

ทำไมต้องทำ?

- `dp[r][c]` = ราคาที่ถูกที่สุดที่จะไปถึงช่อง (r,c)
- เอาไว้เก็บผลลัพธ์ของเส้นทางที่ดีที่สุด

4 กำหนดจุดเริ่มต้น

```
dp[0][0] = grid[0][0]
```

ความหมาย:

- ช่องแรก (0,0) ต้องเสีย = เท่ากับค่าของช่องแรก
- ถ้า `grid[0][0] = 1` แล้ว `dp[0][0] = 1` ตัวอย่าง:

grid:	dp:
[1, 3, 1]	[1, ?, ?]
[1, 5, 1] →	[?, ?, ?]
[4, 2, 1]	[?, ?, ?]

5 คำนวณแถวแรก (บรรทัดที่ 1)

```
for c in range(1, cols):
    dp[0][c] = dp[0][c-1] + grid[0][c]
```

ความหมาย: แถวแรก เดินได้แค่ขวาเท่านั้น

- `c-1` = ช่องด้านซ้าย
- `dp[0][c-1] + grid[0][c]` = ราคาสะสมของด้านซ้าย + ราคาช่องนี้

ตัวอย่าง:

grid:	dp:
[1, 3, 1]	[1, 4, 5]
[1, 5, 1] →	[?, ?, ?]
[4, 2, 1]	[?, ?, ?]

```
dp[0][1] = dp[0][0] + grid[0][1] = 1 + 3 = 4    dp[0][2] = dp[0][1] + grid[0][2] = 4 + 1 = 5
```

ทำไมแบบนี้? เพราะจากช่อง (0,0) ไปช่อง (0,1) ต้องเดินขวา ต้องเสียทั้ง (0,0) และ (0,1)

6 คำนวณหลักแรก (บรรทัดที่ 2)

```
for r in range(1, rows):
    dp[r][0] = dp[r-1][0] + grid[r][0]
```

ความหมาย: หลักแรก เดินได้แค่ลงเท่านั้น

- `r-1` = ช่องด้านบน
- `dp[r-1][0] + grid[r][0]` = ราคาสะสมของด้านบน + ราคาช่องนี้

ตัวอย่าง:

grid:	dp:
[1, 3, 1]	[1, 4, 5]
[1, 5, 1] →	[2, ?, ?]
[4, 2, 1]	[6, ?, ?]

`dp[1][0] = dp[0][0] + grid[1][0] = 1 + 1 = 2` `dp[2][0] = dp[1][0] + grid[2][0] = 2 + 4 = 6`

7 คำนวณส่วนที่เหลือ (บรรทัดที่ 3)

```
for r in range(1, rows):
    for c in range(1, cols):
        dp[r][c] = min(dp[r-1][c], dp[r][c-1]) + grid[r][c]
```

นี่คือส่วนที่สำคัญที่สุด!

ความหมาย: ทุกช่อง เดินได้ 2 ทาง: จากบน หรือ จากซ้าย

- `dp[r-1][c]` = ราคาสะสมถ้ามาจากบน
- `dp[r][c-1]` = ราคาสะสมถ้ามาจากซ้าย
- `min()` = เลือกเส้นทางที่ถูกกว่า
- `+ grid[r][c]` = บวกราคาช่องปัจจุบัน

ตัวอย่าง: คำนวณ `dp[1][1]`

1. ราคาจากบน = `dp[0][1] = 4`
2. ราคาจากซ้าย = `dp[1][0] = 2`
3. เลือกทางถูกกว่า: `min(4, 2) = 2` ← ถูกที่สุด (มาจากซ้าย)
4. บวกราคาช่องนี้: `dp[1][1] = 2 + grid[1][1] = 2 + 5 = 7`

ทำไมต้อง `min()`? เพราะเราต้องการเส้นทางที่ ราคาน้อยที่สุด ไม่ใช่มากที่สุด

8 ส่วนสุดท้าย: คืนคำตอบ

```
return dp[rows-1][cols-1]
```

ความหมาย:

- `rows-1` = แถวสุดท้าย (ถ้า 3 แถว คือ แถว 2)
- `cols-1` = หลักสุดท้าย (ถ้า 3 หลัก คือ หลัก 2)

- `dp[rows-1][cols-1]` = ราคาสะสมของจุดหมาย

ตัวอย่าง:

```
dp ที่เสร็จสมบูรณ์:  
[1, 4, 5]  
[2, 7, 6]  
[6, 8, 7] ← คำนวณนี้ (7)  
  ↑  
  จุดหมาย
```

ตัวอย่างสมบูรณ์ (ทีละขั้นตอน)

```
grid = [  
    [1, 3, 1],  
    [1, 5, 1],  
    [4, 2, 1]  
]
```

- **Step 1:** ตรวจสอบ (ข้ามเพราะ grid ไม่ว่าง)
- **Step 2:** นับแถวหลัก: `rows = 3, cols = 3`
- **Step 3:** สร้าง dp ว่างๆ: `dp = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]`
- **Step 4:** กำหนดจุดเริ่มต้น: `dp[0][0] = grid[0][0] = 1`
- **Step 5:** คำนวณแถวแรก: `c=1: dp[0][1] = 4; c=2: dp[0][2] = 5`
- **Step 6:** คำนวณหลักแรก: `r=1: dp[1][0] = 2; r=2: dp[2][0] = 6`
- **Step 7:** คำนวณช่องที่เหลือ:
 - `dp[1][1]: min(4, 2) + 5 = 7`
 - `dp[1][2]: min(5, 7) + 1 = 6`
 - `dp[2][1]: min(7, 6) + 2 = 8`
 - `dp[2][2]: min(6, 8) + 1 = 7`
- **Step 8:** ผลลัพธ์สุดท้าย: ค่า `dp[2][2] = 7` 

ทำไมวิธีนี้ดี?

- ❌ วิธี "ลองทั้งหมด" (Brute Force): เลือกเส้นทางทั้งหมด → ซ้ำมาก! (3×3 grid: 1,024 วิธี; 100×100 : ไม่ไหว!)
- ✅ วิธี "บันทึก" (Dynamic Programming): จำเส้นทางที่ดีที่สุดที่ได้ลอง → ไว! (100×100 : 10,000 ขั้นตอน ได้เร็ว!)

 ตาราง dp คืออะไร? ลองนึกภาพ: คุณเล่นเกม "ค้นหาเส้นทาง" ที่มี checkpoint เพื่อเก็บตัวเลขที่ดีที่สุดไว้

? FAQ

- Q: ทำไมต้อง `range(1, cols)` ไม่ใช่ `range(0, cols)`? เพราะ `[0][0]` ได้กำหนดแล้ว ถ้าเริ่มจาก 0 จะทับมัน
- Q: ทำไม `min()` ไม่ใช่ `max()`? เพราะเราต้องเส้นทางที่เสีย น้อยที่สุด ไม่ใช่มากที่สุด
- Q: ทำไม `rows-1` ไม่ใช่ `rows`? เพราะนับจาก 0 ถ้ามี 3 แถว คือ 0,1,2 สุดท้ายคือ `2 = 3-1`

สรุป

ส่วน	ทำไม
------	------

if not grid	ป้องกัน error ถ้า grid ว่าง
[[0...]]	เตรียมตารางเก็บผลลัพธ์
rows-1, cols-1	ตำแหน่งช่องสุดท้าย (นับจาก 0)
for หลายรอบ	ต้องคำนวณทุกช่องที่ละตัว
min()	เลือกเส้นทางถูกที่สุด
c-1, r-1	มาจากช่องซ้ายหรือบน

ตอนนี้เข้าใจแล้วไหม?